

#중급반 7주차

최단 경로

<Dijkstra's Algorithm>

2p /<최단 경로>

3p /<동작 원리>

20p /<구현 방법>

22p /<연습 문제>

#다익스트라 알고리즘

<Dijkstra's Algorithm>

정해진 하나의 시작 정점에서부터
다른 모든 정점으로 가는 최단 경로를 구하는 알고리즘

1. 음수 가중치 간선이 존재할 경우 사용할 수 없어요.
2. 시간 복잡도는 $O(E \log V)$ / (E = Edge, V = Vertex)

#다익스트라 알고리즘

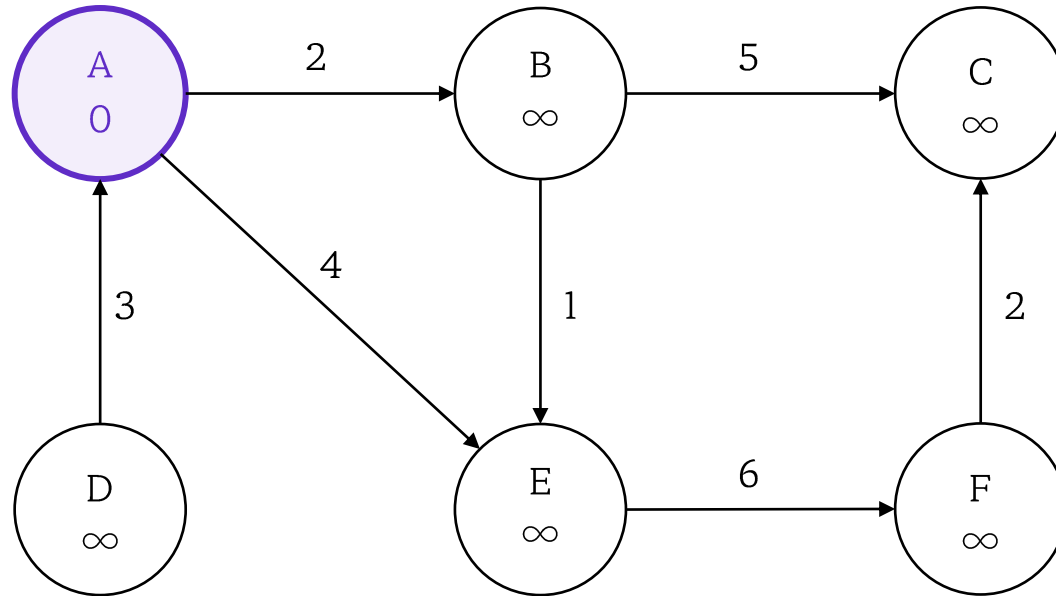
<Dijkstra's Algorithm> - 동작 원리

1. 시작 노드로부터 다른 노드 사이의 최단 거리를 모두 INF로 설정한다.
2. 방문하지 않은 노드 중 시작 노드와의 거리가 가장 짧은 노드에 방문한다.
3. 방문한 노드와 인접한 노드의 최단 거리를 갱신한다.
4. 모든 노드에 방문할 때까지 2~3을 반복한다.

#다익스트라 알고리즘

<Dijkstra's Algorithm> - 동작 원리

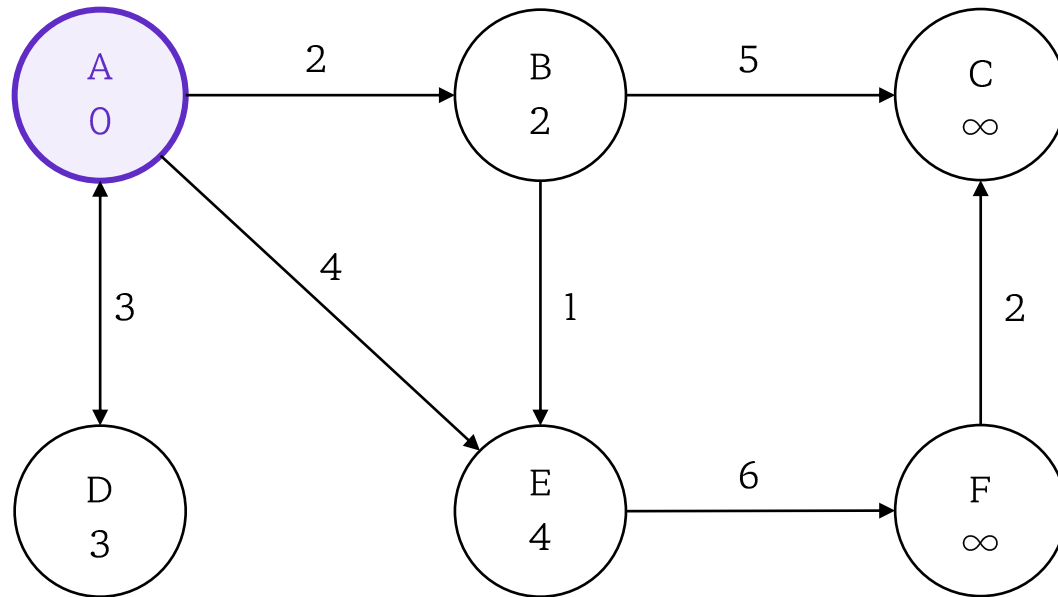
1. 출발 정점 A에서 출발 정점 A까지의 거리를 0으로 확정한다.



#다익스트라 알고리즘

<Dijkstra's Algorithm> - 동작 원리

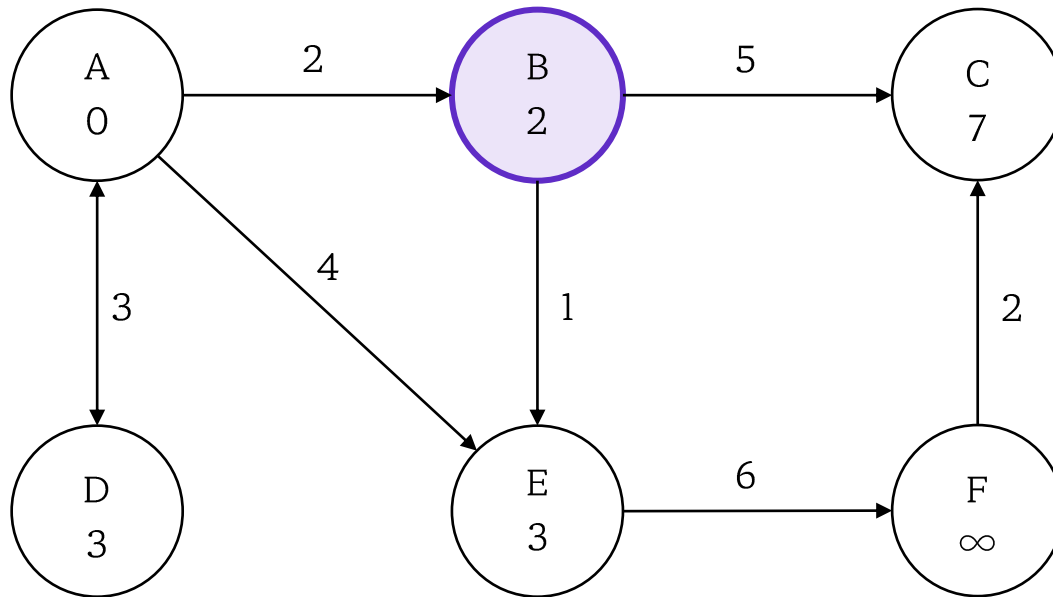
1. 방문하지 않은 노드 중 거리가 가장 짧은 노드인 A에 방문한다.
2. A와 인접한 노드의 거리를 갱신한다.
3. A를 방문한 노드로 처리한다.



#다익스트라 알고리즘

<Dijkstra's Algorithm> - 동작 원리

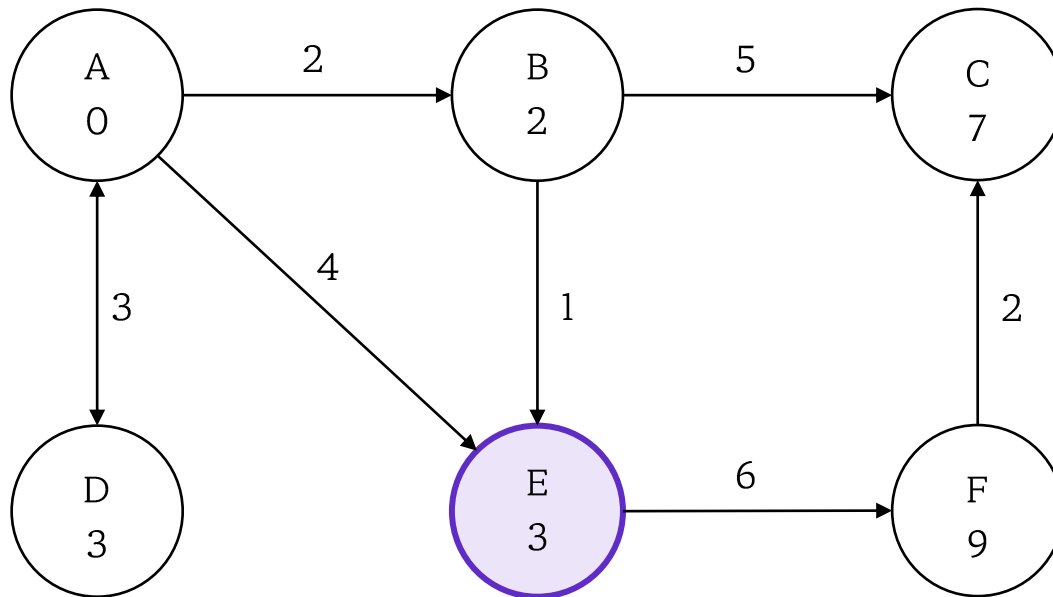
1. 방문하지 않은 노드 중 거리가 가장 짧은 노드에 방문한다.
2. 방문한 노드와 인접한 노드의 거리를 갱신한다.
3. 선택한 노드를 방문한 노드로 처리한다.



#다익스트라 알고리즘

<Dijkstra's Algorithm> - 동작 원리

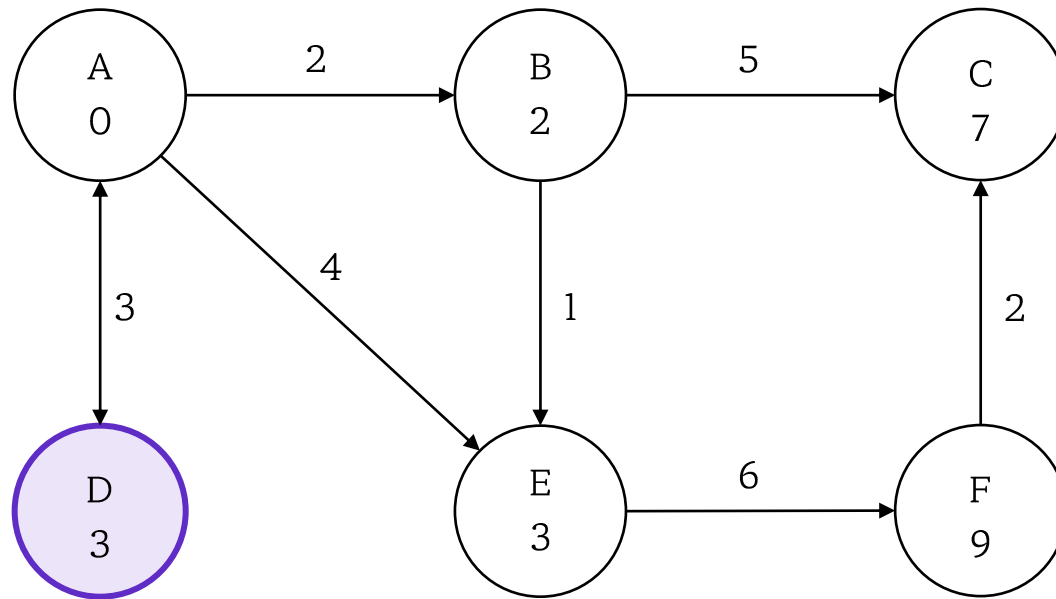
1. 방문하지 않은 노드 중 거리가 가장 짧은 노드에 방문한다.
2. 방문한 노드와 인접한 노드의 거리를 갱신한다.
3. 선택한 노드를 방문한 노드로 처리한다.



#다익스트라 알고리즘

<Dijkstra's Algorithm> - 동작 원리

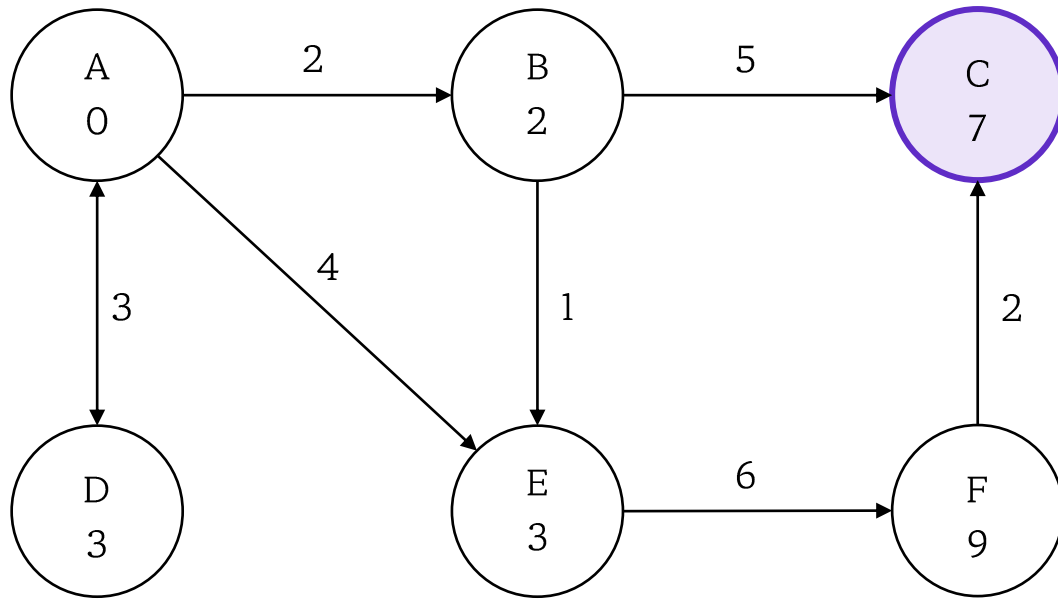
1. 방문하지 않은 노드 중 거리가 가장 짧은 노드에 방문한다.
2. 방문한 노드와 인접한 노드의 거리를 갱신한다.
3. 선택한 노드를 방문한 노드로 처리한다.



#다익스트라 알고리즘

<Dijkstra's Algorithm> - 동작 원리

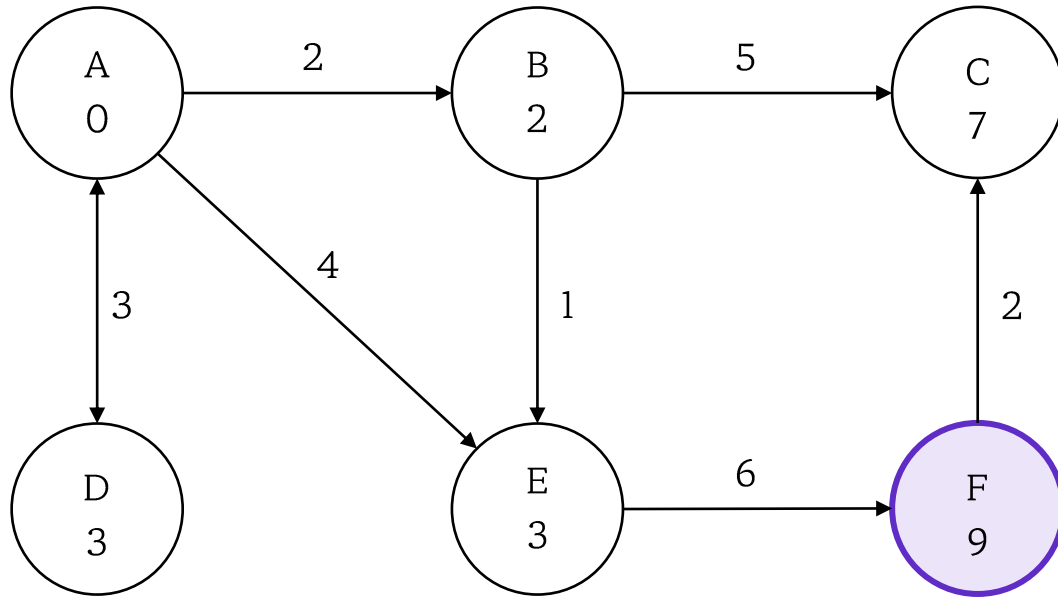
1. 방문하지 않은 노드 중 거리가 가장 짧은 노드에 방문한다.
2. 방문한 노드와 인접한 노드의 거리를 갱신한다.
3. 선택한 노드를 방문한 노드로 처리한다.



#다익스트라 알고리즘

<Dijkstra's Algorithm> - 동작 원리

1. 방문하지 않은 노드 중 거리가 가장 짧은 노드에 방문한다.
2. 방문한 노드와 인접한 노드의 거리를 갱신한다.
3. 선택한 노드를 방문한 노드로 처리한다.



#다익스트라 알고리즘

<Dijkstra's Algorithm> - 동작 원리

1. 시작 노드로부터 다른 노드 사이의 최단 거리를 모두 INF로 설정한다.
2. 방문하지 않은 노드 중 시작 노드와의 거리가 가장 짧은 노드에 방문한다.
3. 방문한 노드와 인접한 노드의 최단 거리를 갱신한다.
4. 모든 노드에 방문할 때까지 2~3을 반복한다.

가장 짧은 노드를 빠르게 찾기 위해, '우선순위 큐' 자료구조를 사용합니다.

#다익스트라 알고리즘

<Dijkstra's Algorithm> - 동작 원리

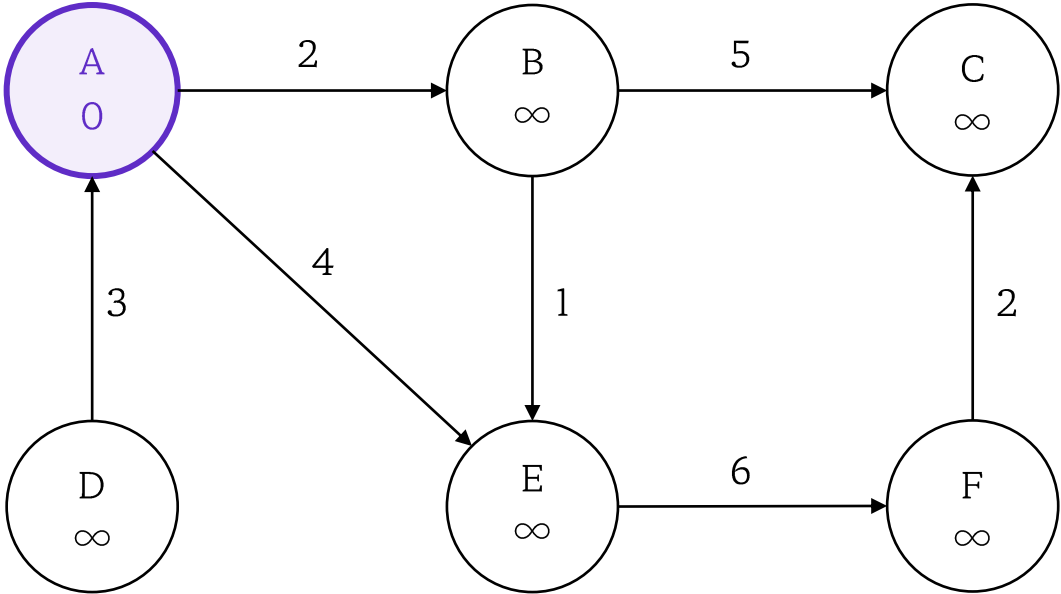
BFS가 정점 간의 최단거리를 구할 때 사용된다는 것이 기억나시나요?

다익스트라 알고리즘은 쉽게 생각하면 BFS에서 큐 대신 우선순위 큐를 사용해 Greedy하게 간선의 가중치(거리)를 처리함으로써 효율적으로 최단경로를 확정하는 알고리즘입니다.

#다익스트라 알고리즘

<Dijkstra's Algorithm> - 동작 원리

1. 출발 정점 A에서 출발 정점 A까지의 거리를 0으로 확정한다.



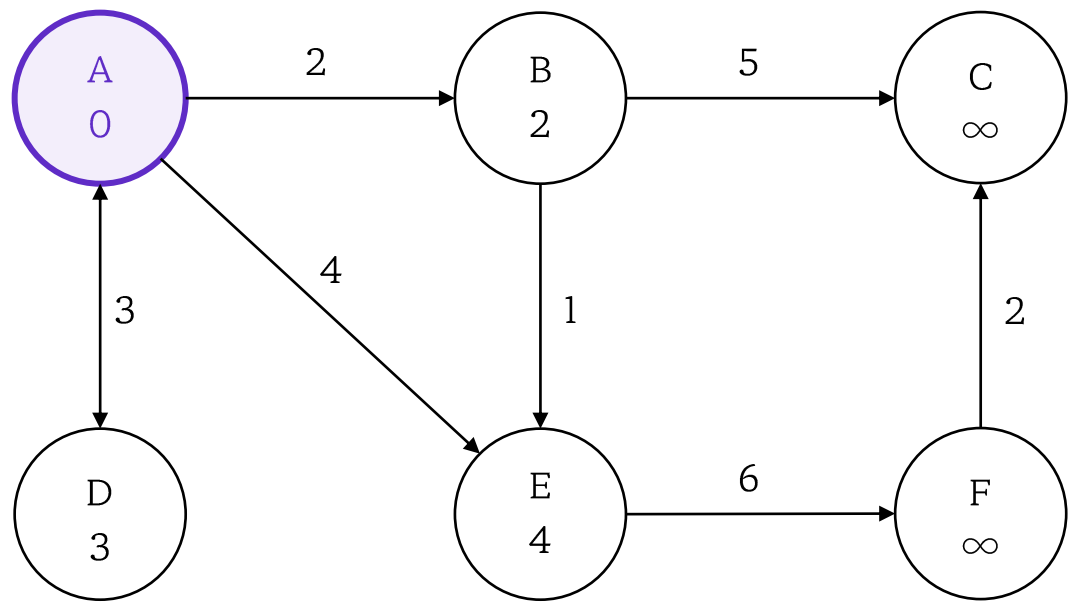
<Priority Queue>

From	To	Distance
A	A	0

#다익스트라 알고리즘

<Dijkstra's Algorithm> - 동작 원리

- 1. 방문하지 않은 노드 중 거리가 가장 짧은 노드인 A에 방문한다.
- 2. A와 인접한 노드의 거리를 갱신한다.
- 3. A를 방문한 노드로 처리한다.



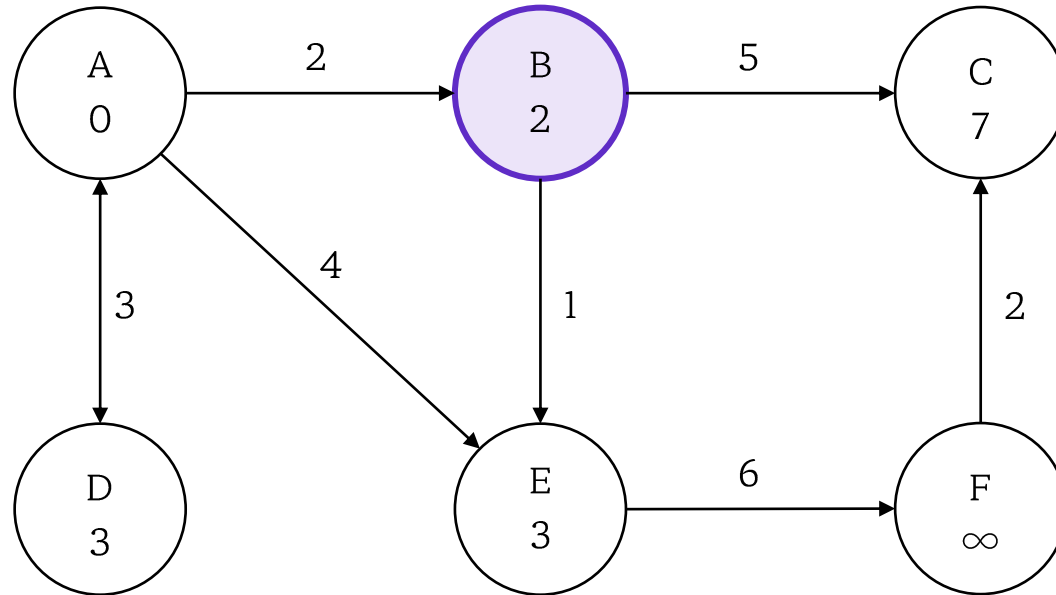
<Priority Queue>

From	To	Distance
A	B	2
A	D	3

#다익스트라 알고리즘

<Dijkstra's Algorithm> - 동작 원리

1. 방문하지 않은 노드 중 거리가 가장 짧은 노드에 방문한다.
2. 방문한 노드와 인접한 노드의 거리를 갱신한다.
3. 선택한 노드를 방문한 노드로 처리한다.



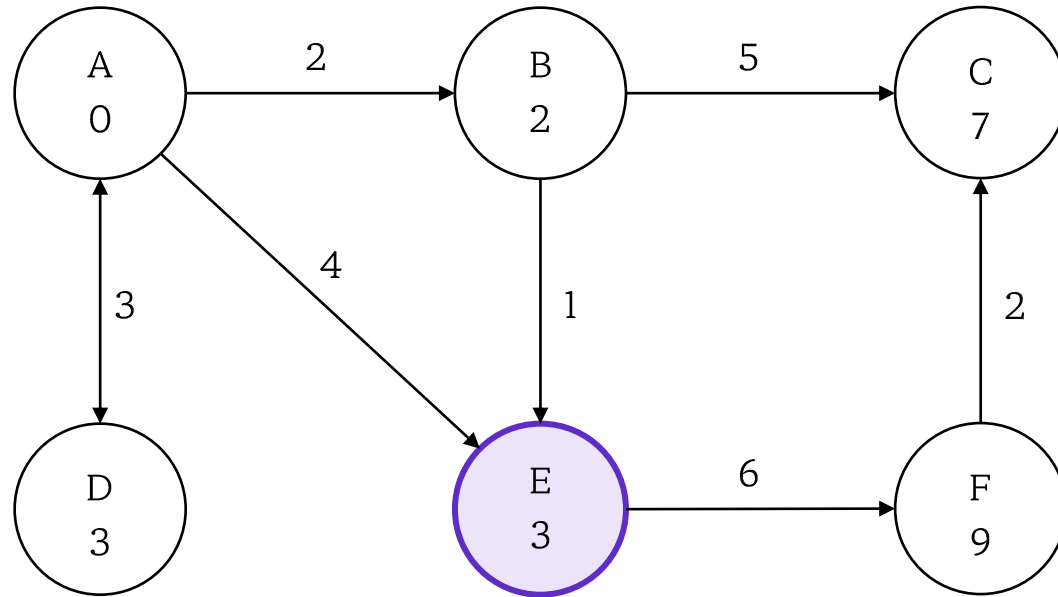
<Priority Queue>

From	To	Distance
B	E	3
A	D	3
B	C	7

#다익스트라 알고리즘

<Dijkstra's Algorithm> - 동작 원리

1. 방문하지 않은 노드 중 거리가 가장 짧은 노드에 방문한다.
2. 방문한 노드와 인접한 노드의 거리를 갱신한다.
3. 선택한 노드를 방문한 노드로 처리한다.



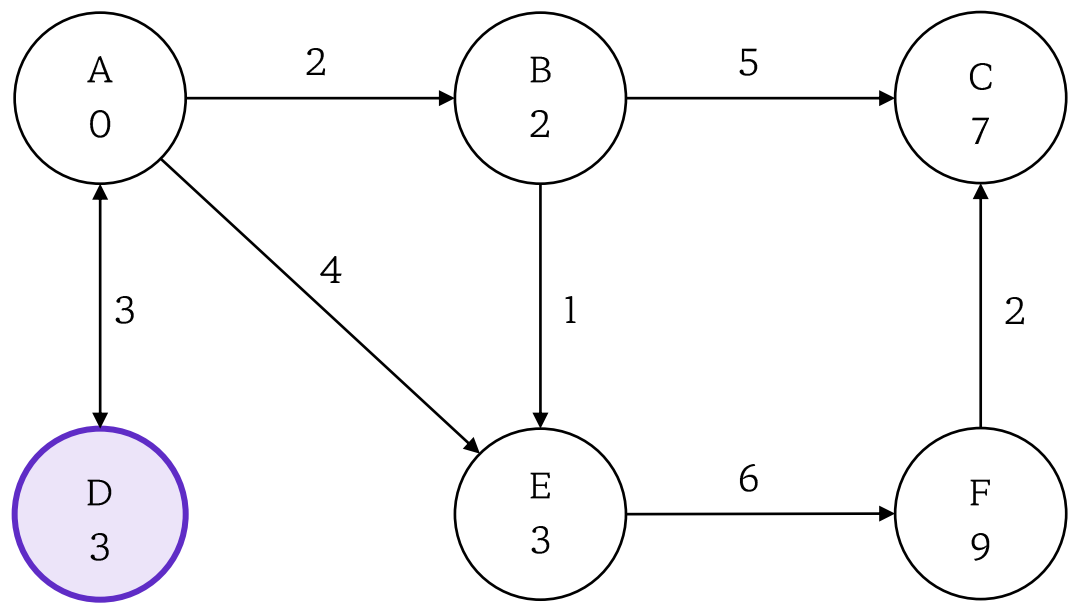
<Priority Queue>

From	To	Distance
A	D	3
B	C	7
E	F	9

#다익스트라 알고리즘

<Dijkstra's Algorithm> - 동작 원리

- 1. 방문하지 않은 노드 중 거리가 가장 짧은 노드에 방문한다.
- 2. 방문한 노드와 인접한 노드의 거리를 갱신한다.
- 3. 선택한 노드를 방문한 노드로 처리한다.



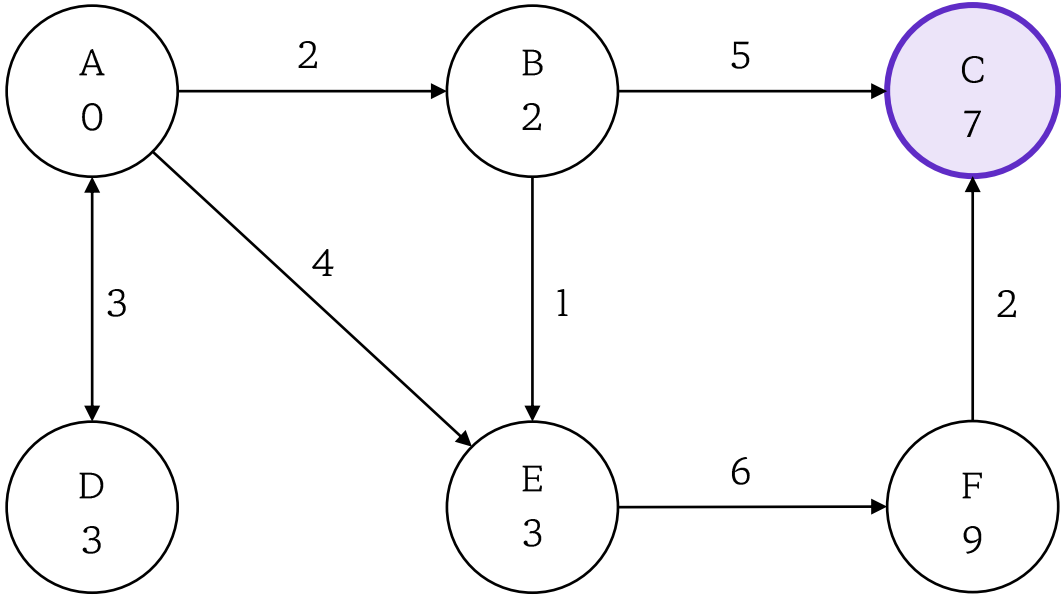
<Priority Queue>

From	To	Distance
B	C	7
E	F	9

#다익스트라 알고리즘

<Dijkstra's Algorithm> - 동작 원리

- 1. 방문하지 않은 노드 중 거리가 가장 짧은 노드에 방문한다.
- 2. 방문한 노드와 인접한 노드의 거리를 갱신한다.
- 3. 선택한 노드를 방문한 노드로 처리한다.



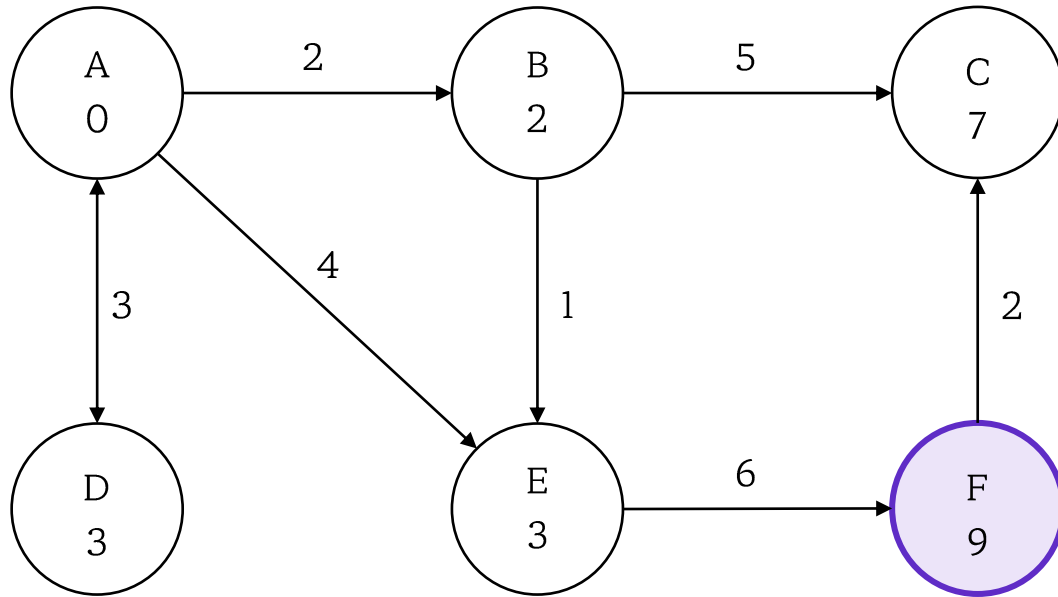
<Priority Queue>

From	To	Distance
E	F	9

#다익스트라 알고리즘

<Dijkstra's Algorithm> - 동작 원리

1. 방문하지 않은 노드 중 거리가 가장 짧은 노드에 방문한다.
2. 방문한 노드와 인접한 노드의 거리를 갱신한다.
3. 선택한 노드를 방문한 노드로 처리한다.



<Priority Queue>

From	To	Distance

#다익스트라 알고리즘

<Dijkstra's Algorithm> - 구현 코드

1. 시작 노드로부터 다른 노드 사이의 최단 거리를 모두 INF로 설정한다.
2. 모든 노드에 방문할 때까지 다음을 반복한다.
 1. 방문하지 않은 노드 중 시작 노드와의 거리가 가장 짧은 노드에 방문한다.
 2. 방문한 노드와 인접한 노드의 최단 거리를 갱신한다.
 3. 선택한 노드를 방문 완료 상태로 바꾼다.

```
// vector<int> dist(V+1, INF), K: 시작하는 정점의 번호;
void dijkstra(vector<vector<pair<int, int>>>& G, vector<int>& dist, int K) {
    priority_queue<pair<int, int>> PQ;
    PQ.push({0, K});
    dist[K] = 0;

    while(!PQ.empty()){
        int cost = -PQ.top().first;
        int curr = PQ.top().second;
        PQ.pop();

        for(int i=0; i < G[curr].size(); i++){
            int next = G[curr][i].first;
            int weight = G[curr][i].second;
            if(dist[next] > cost + weight){
                dist[next] = cost + weight;
                PQ.push({-dist[next], next});
            }
        }
    }
}
```

#다익스트라 알고리즘

<Dijkstra's Algorithm> - 정당성 증명

다익스트라 알고리즘은 항상 가장 가까운 정점을 먼저 처리하므로,

이미 처리된 정점까지의 거리는 최단거리임이 보장됩니다.

간선의 가중치가 모두 0 이상이기 때문에, 나중에 더 짧은 경로가 생길 수 없기 때문입니다.

따라서 **매번 가장 짧은 경로를 확정하며** 나아가는 방식이

전체 그래프에 대해 **올바른 최단거리를 보장합니다.**

#다익스트라 알고리즘

<Dijkstra's Algorithm> - 연습 문제

3 최단경로 (JUNGOL #1941)

<문제 설명>

- 정점 N 개와 간선 M 개로 이루어진 그래프가 주어진다.
- 각 간선에는 **가중치**가 존재한다.
- 1번 정점에서 N 번 정점까지의 최단거리를 구하기

<제약 조건>

- $1 \leq N \leq 20,000$
- $1 \leq M \leq 100,000$
- 간선의 가중치는 10^4 이하 양의 정수

#다익스트라 알고리즘

<Dijkstra's Algorithm> - 연습 문제

3 최단경로 (JUNGOL #1941)

<문제 해설>

다익스트라 알고리즘을 그대로 적용하는 문제

#다익스트라 알고리즘

<Dijkstra's Algorithm> - 연습 문제

3 지하철 (JUNGOL #2097)

<문제 설명>

- 정점 N 개와 간선 N^2 개로 이루어진 그래프가 주어진다.
- 각 간선에는 **가중치**가 존재한다.
- 1번 정점에서 M 번 정점으로 가는 최단 경로 구하기
- **최소 비용 뿐 아니라 경로도 출력해야 한다.**

<제약 조건>

- $3 \leq N \leq 100$
- $1 \leq M \leq N$
- $0 \leq (\text{간선의 가중치}) \leq 100$

#다익스트라 알고리즘

<Dijkstra's Algorithm> - 연습 문제

3 지하철 (JUNGOL #2097)

<문제 해설>

정점 k의 최단 거리를 갱신할 때, k로 가기 이전의 노드를 prev[] 배열에 저장해 둡니다.

M번 정점에서 시작해서 1번 정점이 나올 때까지 prev[] 배열을 따라가주면 됩니다.

```
for(int i=0; i < G[curr].size(); i++){
    int next = G[curr][i].first;
    int weight = G[curr][i].second;

    if(dist[next] > cost + weight){
        dist[next] = cost + weight;
        PQ.push({-dist[next], next});
        prev[next] = curr;
    }
}
```

#필수 문제

3 최단경로 (JUNGOL #1941)

다익스트라 알고리즘 익히기

3 지하철 (JUNGOL #2097)

다익스트라 알고리즘으로 경로 구하기

#연습 문제 도전

3 페이지 전환 (JUNGOL #1108)

다익스트라 여러 번 돌리기

2 소는 왜 길을 건너갔을까? 7 (JUNGOL #4013)

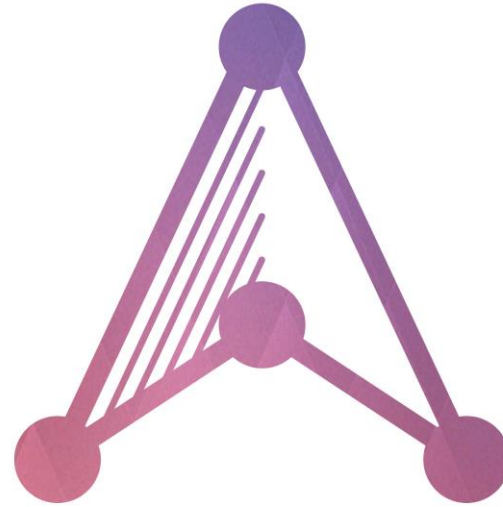
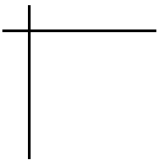
‘상태’를 그래프에 녹여내는 방법

2 꿀꿀이 축제 (JUNGOL #2109)

각 정점에서 X번 정점까지 갈 때, 각각의 거리를 구하기

5 주유소(고등) (JUNGOL #2999)

이전에 방문한 노드의 기름값도 중요하다



A L O H A
The algorithm club.

